Class Attributes
Class Methods

Attributes

- An **attribute** is a variable bound to a class or its instances.
- Access with dot notation:

Syntax
`ClassName.attr` `obj.attr`

Class Attributes

- Defined in the **class body**, **outside methods**.
- Shared by **all instances** of that class.

Python

```python
class Car:
    wheels = 4          # class attribute (same for all cars)
    total_cars = 0      # class attribute (counter)

    def __init__(self, brand, color):
        self.brand = brand     # instance attribute
        self.color = color     # instance attribute
        Car.total_cars += 1
```

Class Attributes

Access

```
print(Car.wheels)          # 4
c1 = Car("Tesla", "Red")
print(c1.wheels)           # 4 (from class)
```

Modify

```
Car.wheels = 6
print(c1.wheels)           # 6 (if not overridden in instance)
```

Instance Attributes

- **Unique** to each object.
- Usually defined in **__init__** with **self**.

Python

```python
c1 = Car("Tesla", "Red")
c2 = Car("BMW", "Blue")
print(c1.brand, c1.color)   # Tesla Red
print(c2.brand, c2.color)   # BMW Blue
```

Changes to one object's instance attributes don't affect others.

Methods

A **method** is a function defined **inside a class**.

We usually **categorize** them as:

- Instance methods
- Class methods
- Static methods

Instance Method (default kind)

First parameter: **self**

Can access:

- instance attributes (**self.x**)
- class attributes (**ClassName.y** or **self.__class__.y**)

Python

```python
class Car:
    wheels = 4
    total_cars = 0

    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
        Car.total_cars += 1

    def info():
        print(f"{self.color} {self.brand}")
```

Class Method

Idea: bound to the class, not the individual object.

There are two ways to create class methods in Python -

- Using **@classmethod** Decorator
- Using **classmethod()** Function

- Using **@classmethod** Decorator

Python	Usage

`class Book:`
`  total_books = 0`
 	
`  def __init__(self, title, author, lang):`
`  self.title = title`
`  self.author = author`
`  self.lang = lang`
`  Book.total_books += 1`
 	
`  @classmethod`
`  def show_count(cls):`
`  print(cls.total_books)`
	`b1 = Book("1984", "Orwell")`
	
	`b2 = Book.from_string("Dune;Herbert;EN")`
	
	`Book.show_count()`
	`# uses cls → sees updated total_books`

- Using **classmethod() function** (less common)

Python

```python
class Book:
    total_books = 0

    def show_count(cls):
        print(cls.total_books)

    show_count = classmethod(show_count)
```

Usage

```python
Book.show_count()
# uses cls → sees updated total_books
```

Static method

In Python, a **static method** is:

- A function defined inside a class.
- Does not take **self** (instance) or **cls** (class) as the first argument.

When to use?
When the behavior is **logically related** to the class, but **doesn't** need access to **instance attributes** or **class attributes** (unless referenced manually)

There are **two** ways to define static method:

- Using **@staticmethod** (recommended)
- Using **Staticmethod()**

Using @staticmethod Decorator

Python

```python
class MathTools:

    @staticmethod
    def add(a, b):
        return a + b

print(MathTools.add(2, 3))   # 5
m = MathTools()
print(m.add(4, 6))           # 10 (works, but still static)
```

Using staticmethod() Function

Python

```python
class MathTools:

    def add(a, b):
        return a + b

    add = staticmethod(add)

print(MathTools.add(2, 3))   # 5
```

When to use what

- **Instance method**
Use when logic needs **object data** (self).

- **Class method**
Use when logic works on the **class itself**: shared state, configs, **alternate constructors**.

- **Static method**
Use when logic is a **utility** related to the class but needs **no self or cls**.

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Encapsulation